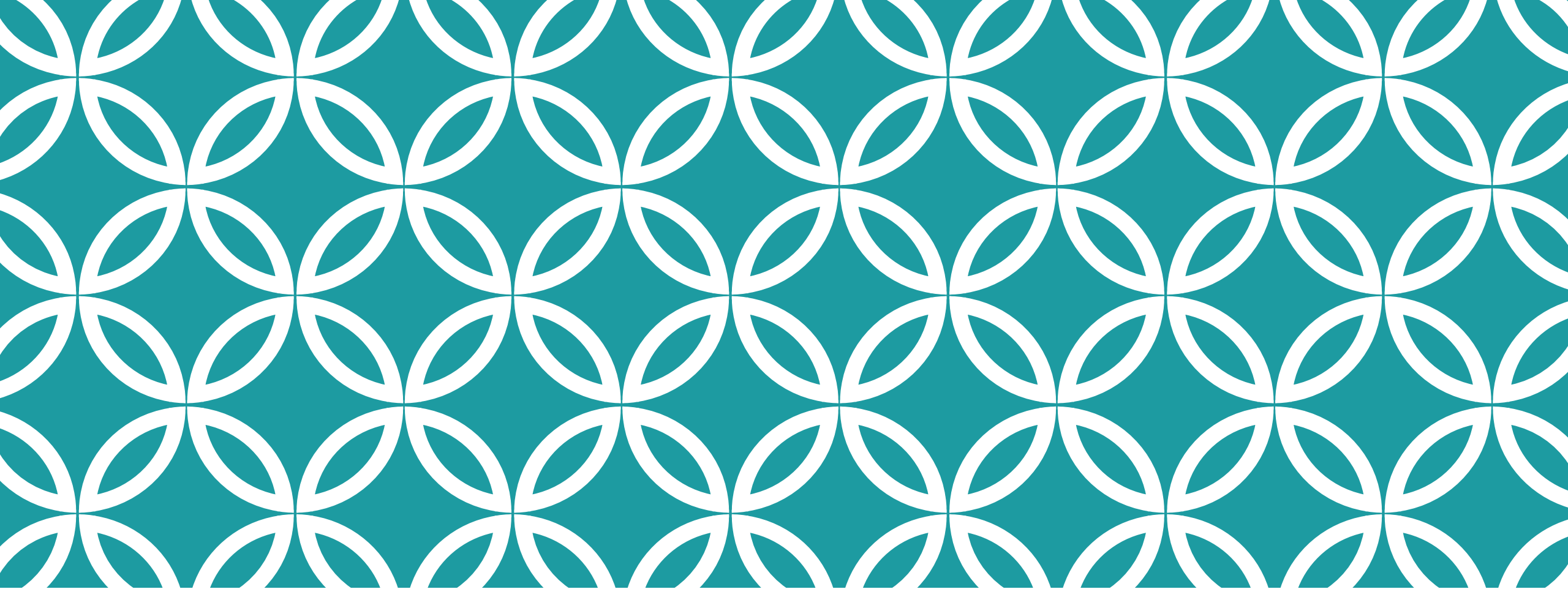




MODULE 4- ADVANCED SQL WITH INTEGRITY, SECURITY AND AUTHORIZATION

Compiled by: Mrs. Jisha Tinsu

NESTED SUB QUERIES

- A subquery can be placed in a number of SQL clauses like WHERE clause, FROM clause, HAVING clause.
- You can use Subquery with SELECT, UPDATE, INSERT, DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN, etc.
- A subquery is a query within another query. The outer query is known as the main query, and the inner query is known as a subquery.
- Subqueries are on the right side of the comparison operator.
- A subquery is enclosed in parentheses.
- In the Subquery, ORDER BY command cannot be used. But GROUP BY command can be used to perform the same function as ORDER BY command.

1. SUBQUERIES WITH THE SELECT STATEMENT

- SQL subqueries are most frequently used with the Select statement.

- **Syntax**

SELECT column_name

FROM table_name

WHERE column_name expression operator

(SELECT column_name from table_name WHERE ...);

2. SUBQUERIES WITH THE INSERT STATEMENT

- SQL subquery can also be used with the Insert statement. In the insert statement, data returned from the subquery is used to insert into another table.

- **Syntax:**

```
INSERT INTO table_name (column1, column2, column3....)  
SELECT *  
FROM table_name  
WHERE VALUE OPERATOR
```

EXAMPLE:

- Suppose you have two tables: **employees and departments.**
- You want to insert employees into a managers table if they are in a specific department, for example, the "Sales" department.

SQL Statement:

```
INSERT INTO managers (manager_id, manager_name,  
department_id)  
SELECT employee_id, first_name || ' ' || last_name, department_id  
FROM employees  
WHERE department_id = (  
    SELECT department_id  
    FROM departments  
    WHERE department_name = 'Sales'  
);
```

EXAMPLE:

■ Nested Query (Subquery):

```
SELECT department_id  
FROM departments  
WHERE department_name = 'Sales'
```

finds the department_id for the
"Sales" department.

■ Outer Query:

```
SELECT employee_id, first_name || ' ' || last_name, department_id  
FROM employees  
WHERE department_id = (...)
```

selects the employee_id, the
full name (first_name || ' '
|| last_name), and
department_id from the
employees table where the
department_id matches the
result from the subquery (i.e.,
the "Sales" department).

■ INSERT INTO:

The `INSERT INTO managers` statement inserts these selected values into the managers' table.

3. SUBQUERIES WITH THE UPDATE STATEMENT

- The subquery of SQL can be used in conjunction with the Update statement.
- When a subquery is used with the Update statement, then either single or multiple columns in a table can be updated.

- **Syntax**

```
UPDATE table
SET column_name = new_value
WHERE VALUE OPERATOR
      (SELECT COLUMN_NAME
      FROM TABLE_NAME
      WHERE condition);
```

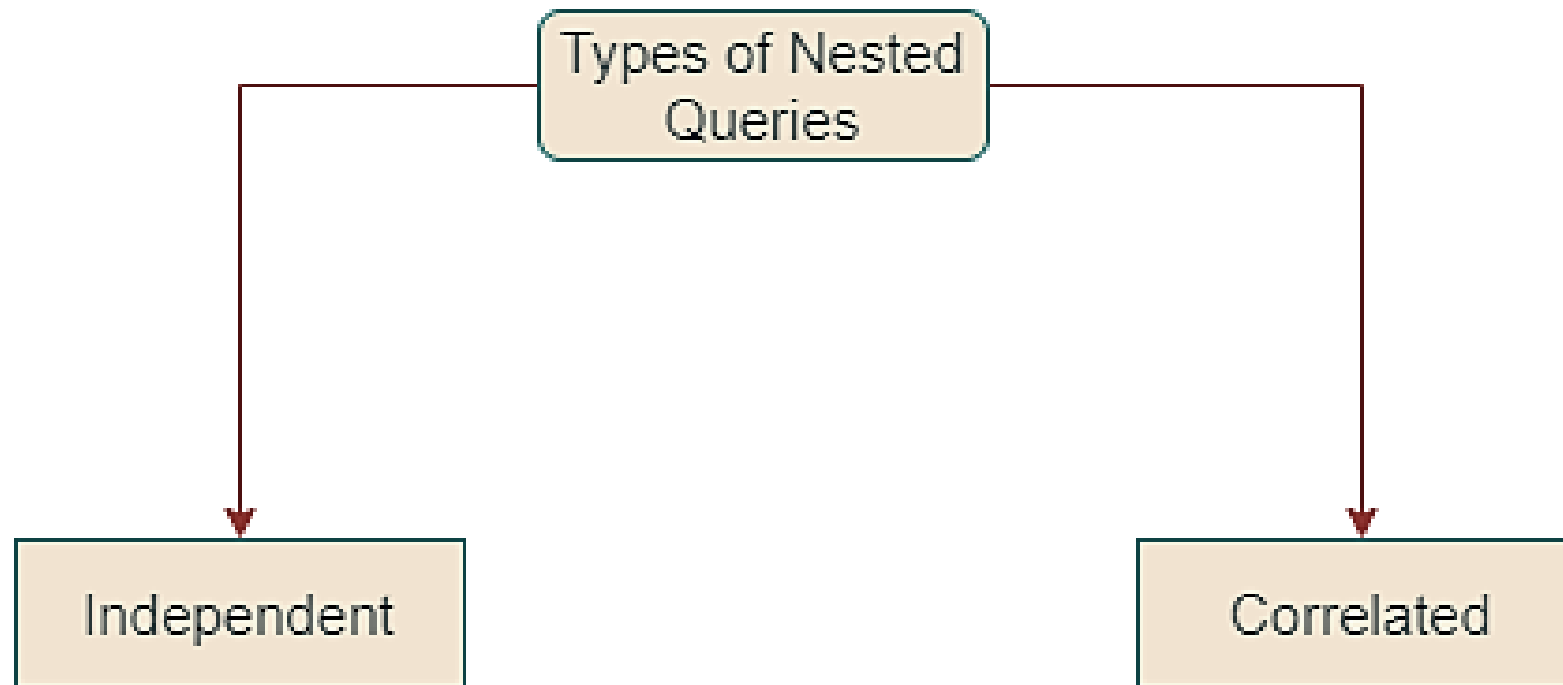
4. SUBQUERIES WITH THE DELETE STATEMENT

- The subquery of SQL can be used in conjunction with the Delete statement just like any other statements mentioned above.

- **Syntax**

```
DELETE FROM TABLE_NAME  
WHERE VALUE OPERATOR  
(SELECT COLUMN_NAME  
FROM TABLE_NAME  
WHERE condition);
```


TYPES OF NESTED QUERIES IN SQL



INDEPENDENT NESTED QUERIES

- Non-correlated (or Independent) Nested Queries : Non-correlated (or Independent) subqueries are executed independently of the outer query. Their results are passed to the outer query.
- Execution Order in Independent Nested Queries: In independent nested queries, the execution order is from the innermost query to the outer query. An outer query won't be executed until its inner query completes its execution. The outer query uses the result of the inner query.

OPERATORS USED IN INDEPENDENT NESTED QUERIES

- **IN Operator:** This operator checks if a column value in the outer query's result is present in the inner query's result. The final result will have rows that satisfy the IN condition.
- **NOT IN Operator:** This operator checks if a column value in the outer query's result is not present in the inner query's result. The final result will have rows that satisfy the NOT IN condition.
- **ALL Operator:** This operator compares a value of the outer query's result with all the values of the inner query's result and returns the row if it matches all the values.
- **ANY Operator:** This operator compares a value of the outer query's result with all the inner query's result values and returns the row if there is a match with any value.

CORRELATED NESTED QUERIES

- Correlated subqueries are executed once for each row of the outer query. They use values from the outer query to return results.
- Execution Order in Co-related Nested Queries: In correlated nested queries, the inner query uses values from the outer query, and the execution order is different from that of independent nested queries.
 1. First, the outer query selects the first row.
 2. Inner query uses the value of the selected row. It executes its query and returns a result set.
 3. Outer query uses the result set returned by the inner query. It determines whether the selected row should be included in the final output.
 4. Steps 2 and 3 are repeated for each row in the outer query's result set.
 5. This process can be resource-intensive. It may lead to performance issues if the query is not optimized properly.

OPERATORS USED IN CO-RELATED NESTED QUERIES

- In co-related nested queries, the following operators can be used:
- **EXISTS Operator:** This operator checks whether a subquery returns any row. If it returns at least one row, EXISTS operator returns true, and the outer query continues to execute. If the subquery returns no row, the EXISTS operator returns false, and the outer query stops execution.
- **NOT EXISTS Operator:** This operator checks whether a subquery returns no rows. If the subquery returns no row, the NOT EXISTS operator returns true, and the outer query continues to execute. If the subquery returns at least one row, the NOT EXISTS operator returns false, and the outer query stops execution.
- **ANY Operator:** This operator compares a value of the outer query's result with one or more values returned by the inner query. If the comparison is true for any one of the values returned by the inner query, the row is included in the final result.
- **ALL Operator:** This operator compares a value of the outer query's result with all the values returned by the inner query. Only if the comparison is true for all the values returned by the inner query, the row is included in the final result.

EXAMPLES

- Consider the following sample table to execute nested queries on these.

- Table: employees table

emp_id	emp_name	dept_id
1	John	1
2	Mary	2
3	Bob	1
4	Alice	3
5	Tom	1

EXAMPLES

- Table: departments table

dept_id	dept_name
1	Sales
2	Marketing
3	Finance

EXAMPLES

■ sales table

sale_id	emp_id	sale_amt
1	1	1000
2	2	2000
3	3	3000
4	1	4000
5	5	5000
6	3	6000
7	2	7000

EXAMPLES

- Example 1: Find the names of all employees in the Sales department.
- Query:

```
SELECT emp_name  
FROM employees  
WHERE dept_id IN (SELECT dept_id  
                  FROM departments  
                  WHERE dept_name = 'Sales');
```

EXAMPLES

- Example 1: Find the names of all employees in the Sales department.

- Query:

```
SELECT emp_name  
FROM employees  
WHERE dept_id IN (SELECT dept_id  
FROM departments  
WHERE dept_name = 'Sales');
```

Output:

emp_name
John
Bob
Tom

EXAMPLES

- Example 2: Find the names of all employees who have made a sale.

```
SELECT emp_name
FROM employees
WHERE EXISTS (SELECT emp_id
              FROM sales
              WHERE employees.emp_id = sales.emp_id);
```

EXAMPLES

- Example 2: Find the names of all employees who have made a sale.

- Query:

```
SELECT emp_name
FROM employees
WHERE EXISTS (SELECT emp_id
              FROM sales
              WHERE employees.emp_id = sales.emp_id);
```

Output:

emp_name
John
Mary
Bob
Tom

EXAMPLES

- Example 3: Find the names of all employees who have made sales greater than \$1000.

Query:

```
SELECT emp_name
FROM employees
WHERE emp_id = ALL (SELECT emp_id
                    FROM sales
                    WHERE sale_amt > 1000);
```

Output:

emp_name
John
Mary
Bob
Tom

NESTED QUERY IN SQL : TABLES

- Consider the following tables:

Student (StudentID, StudentName)

Course (CourseID, CourseName)

S_Course (StudentID, CourseID, Grade)

EXAMPLE: CO-RELATED NESTED QUERIES:

Q. Find the names of students who are enrolled in the course with Course_id = 'C1'.

```
SELECT s. StudentName  
FROM Students s  
WHERE EXISTS  
( SELECT 1   FROM S_Course sc  
WHERE s.StudentId = sc. StudentId AND sc.CourseId = 'C1');
```

EXAMPLE: CO-RELATED NESTED QUERIES:

- Explanation:
- **Outer Query:** Selects the StudentName from the Students table (aliased as s).
- **Correlated Subquery:** The subquery checks whether there is an entry in the S_Course table where:
 - s.Student_id (from the outer query) matches sc.Student_id (in the subquery).
 - sc.Course_id equals 'C1'.
- For each row in the students table, the subquery is evaluated to check if that student is enrolled in the course with Course_id = 'C1'.

COMPLEX QUERIES

- Complex SQL is the use of SQL queries which go beyond the standard SQL of using the SELECT and WHERE commands.
- Complex SQL often involves using complex joins and sub-queries, where queries are nested in WHERE clauses.
- Complex queries frequently involve heavy use of AND and OR clauses.
- These queries make it possible for perform more accurate searches of a database.

1. MULTI-TABLE JOIN WITH AGGREGATION

You have three tables: Orders, Customers, and OrderDetails. You want to find the total amount spent by each customer.

Tables:

Customers: CustomerID, CustomerName

Orders: OrderID, CustomerID, OrderDate

OrderDetails: OrderDetailID, OrderID, ProductID, Quantity, PricePerUnit

1. MULTI-TABLE JOIN WITH AGGREGATION

```
SELECT c.CustomerName, SUM(od.Quantity * od.PricePerUnit)
AS TotalAmountSpent
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY c.CustomerName
ORDER BY TotalAmountSpent DESC;
```

1. MULTI-TABLE JOIN WITH AGGREGATION

- JOIN: Combines the Customers, Orders, and OrderDetails tables.
- SUM: Calculates the total amount spent by each customer.
- GROUP BY: Groups results by CustomerName to calculate the total for each customer.
- ORDER BY: Orders the results by the total amount spent in descending order.

2. SUBQUERY WITH HAVING CLAUSE

- **Tables: Customers: CustomerID, CustomerName**
- **Orders: OrderID, CustomerID, OrderDate**

Scenario: You want to find all customers who have placed more than 5 orders.

2. SUBQUERY WITH HAVING CLAUSE

```
SELECT c.CustomerName, COUNT(o.OrderID) AS  
OrderCount  
FROM Customers c  
JOIN Orders o ON c.CustomerID = o.CustomerID  
GROUP BY c.CustomerName  
HAVING COUNT(o.OrderID) > 5;
```

COUNT: Counts the number of orders for each customer.

HAVING: Filters the results to only include customers who have placed more than 5 orders.

3. CORRELATED SUBQUERY

- **Scenario:** Find employees who earn more than the average salary in their department.

- **Tables:**

- **Employees:** **EmployeeID,** **EmployeeName,** **Salary,**
 DepartmentID

- **Departments:** **DepartmentID, DepartmentName**

3. CORRELATED SUBQUERY

```
SELECT
e.EmployeeName, e.Salary, d.DepartmentName
FROM Employees e
JOIN Departments d
ON e.DepartmentID = d.DepartmentID
WHERE e.Salary > (
    SELECT AVG(e2.Salary)
    FROM Employees e2
    WHERE e2.DepartmentID = e.DepartmentID
);
```

Correlated Subquery: The subquery depends on the outer query. It calculates the average salary for the department of each employee.

WHERE Clause: Filters employees whose salary is greater than the department's average.

VIEW

- Views in SQL are considered as a virtual table. A view also contains rows and columns.
- To create the view, we can select the fields from one or more tables present in the database.
- A view can either have specific rows based on certain condition or all the rows of a table.

Student_Detail

STU_ID	NAME	ADDRESS
1	Stephan	Delhi
2	Kathrin	Noida
3	David	Ghaziabad
4	Alina	Gurugram

Student_Marks

STU_ID	NAME	MARKS	AGE
1	Stephan	97	19
2	Kathrin	86	21
3	David	74	18
4	Alina	90	20
5	John	96	18

1. CREATING VIEW

- A view can be created using the CREATE VIEW statement. We can create a view from a single table or multiple tables.
- **Syntax:**

```
CREATE VIEW view_name AS  
SELECT column1, column2.....  
FROM table_name  
WHERE condition;
```

2. CREATING VIEW FROM A SINGLE TABLE

- Using the tables, we create a View named DetailsView from the table Student_Detail.

- **Query:**

```
CREATE VIEW DetailsView AS  
SELECT NAME, ADDRESS  
FROM Student_Details  
WHERE STU_ID < 4;
```

- Just like table query, we can query the view to view the data.

```
SELECT * FROM DetailsView;
```

2. CREATING VIEW FROM A SINGLE TABLE - EXAMPLE

SELECT * FROM DetailsView; will give the following output.

NAME	ADDRESS
Stephan	Delhi
Kathrin	Noida
David	Ghaziabad

3. CREATING VIEW FROM MULTIPLE TABLES

- View from multiple tables can be created by simply include multiple tables in the SELECT statement.
- In the given example, a view is created named MarksView from two tables Student_Detail and Student_Marks.
- **Query:**

```
CREATE VIEW MarksView AS  
SELECT Student_Detail.NAME, Student_Detail.ADDRESS, Student_Marks.  
MARKS  
FROM Student_Detail, Student_Mark  
WHERE Student_Detail.NAME = Student_Marks.NAME;
```

3. CREATING VIEW FROM MULTIPLE TABLES - EXAMPLE

To display marks :

SELECT * FROM MarksView;

NAME	ADDRESS	MARKS
Stephan	Delhi	97
Kathrin	Noida	86
David	Ghaziabad	74
Alina	Gurugram	90

4. DELETING VIEW

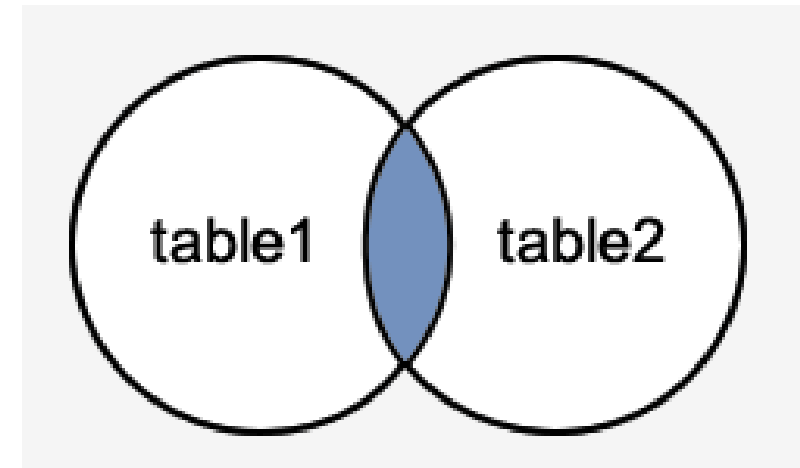
- A view can be deleted using the Drop View statement.

- **Syntax:**

DROP VIEW view_name;

INNER JOIN

```
SELECT columns  
FROM table1  
INNER JOIN table2  
ON table1.column = table2.column;
```



LEFT OUTER JOIN

- The LEFT OUTER JOIN returns all rows from the left hand table specified in the ON condition and only those rows from the other table where the join condition is fulfilled.

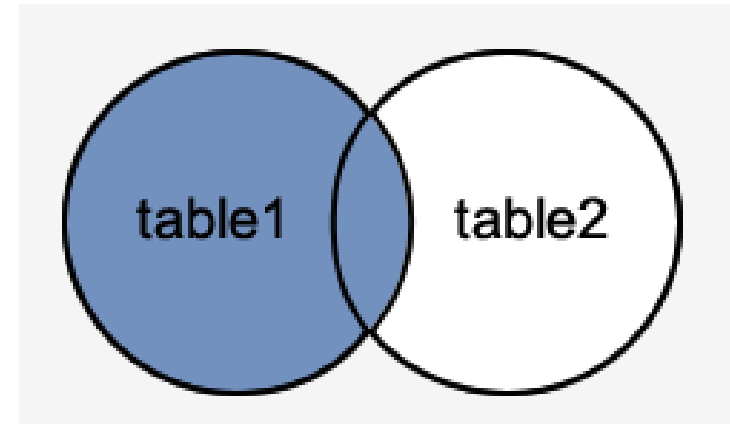
- **Syntax:**

SELECT columns

FROM table1

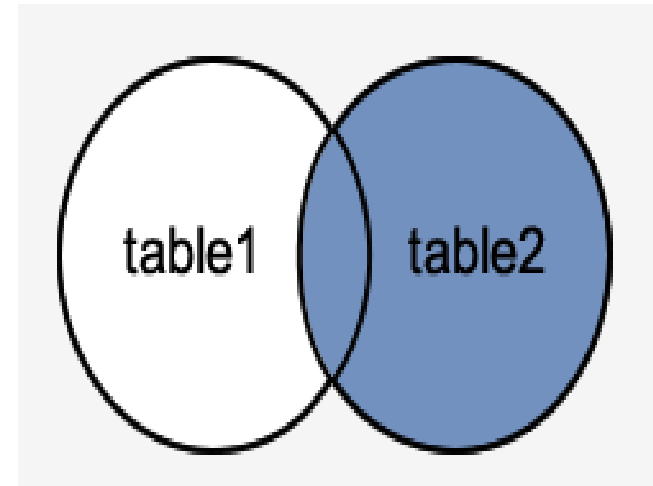
LEFT [OUTER] JOIN table2

ON table1.column = table2.column;



RIGHT OUTER JOIN

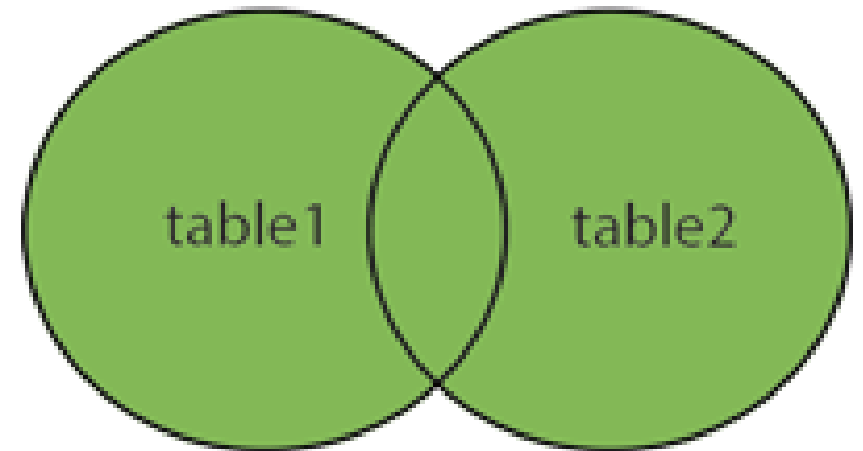
```
SELECT columns  
FROM table1  
RIGHT [OUTER] JOIN table2  
ON table1.column = table2.column;
```



FULL OUTER JOIN

```
SELECT column_name(s)  
FROM table1  
FULL OUTER JOIN table2  
ON table1.column_name = table2.column_name  
WHERE condition;
```

FULL OUTER JOIN



PROCEDURE

- A procedure (often called a stored procedure) is a **collection of pre-compiled SQL statements** stored inside the database.
- It is a subroutine or a subprogram in the regular computing language.
- **A procedure always contains a name, parameter lists, and SQL statements.**
- If we consider the enterprise application, we always need to perform specific tasks such as database cleanup, processing payroll, and many more on the database regularly.
- Such tasks involve multiple SQL statements for executing each task. This process might be easy if we group these tasks into a single task.
- We can fulfill this requirement in SQL by creating a stored procedure in our database.
- A procedure is called a **recursive stored procedure** when it calls itself. Most database systems support recursive stored procedures. But it is not supported well in MySQL.

PROCEDURE

- In SQL Server, a **procedure** is a precompiled collection of one or more SQL statements that can be executed as a unit. Procedures can perform operations like querying, updating, inserting, or deleting data. They are used to encapsulate logic, making it reusable and easier to maintain.

- Types of Procedures:-

- a) System Procedures:

- b) User-Defined Procedures:

TYPES OF PROCEDURES:-

■ System Procedures:

- These are built-in procedures provided by SQL Server.
- They perform administrative and informational activities related to the database system.
- Example: `sp_help` provides information about a database object.

■ User-Defined Procedures:

- These are created by users to perform custom tasks.
- They can include logic for data manipulation, business rules, and more.

STORED PROCEDURES IN SQL SERVER

- A **stored procedure** is a user-defined procedure that is stored in the database and can be invoked by name. It can accept input parameters, return output parameters, and return status codes or result sets.
- **Benefits of Stored Procedures:**
 - **Reusability**: Stored procedures can be used multiple times by different applications or users.
 - **Modularity**: They allow complex logic to be broken down into manageable pieces.
 - **Security**: Permissions can be granted at the procedure level, controlling access to underlying tables.
 - **Performance**: Stored procedures are precompiled and stored in memory, leading to faster execution.
 - **Reduced Network Traffic**: Multiple SQL statements can be encapsulated in a single procedure, reducing the number of network round trips.

STORED PROCEDURE FEATURES

- Stored Procedure increases the performance of the applications. Once stored procedures are created, they are compiled and stored in the database.
- Stored procedure reduces the traffic between application and database server. Because the application has to send only the stored procedure's name and parameters instead of sending multiple SQL statements.
- Stored procedures are reusable and transparent to any applications.
- A procedure is always secure. The database administrator can grant permissions to applications that access stored procedures in the database without giving any permissions on the database tables.

SYNTAX

```
CREATE PROCEDURE procedure_name
@parameter1 datatype = default_value, -- Optional parameters
@parameter2 datatype = default_value -- Optional parameters
AS
BEGIN
    -- SQL statements
    SELECT column1, column2 FROM table_name WHERE condition;
    -- Additional SQL logic
END;
```

Section	Explanation
CREATE PROCEDURE procedure_name	- CREATE PROCEDURE: This keyword initiates the creation of a new stored procedure. - procedure_name: The name you give to the stored procedure. It must be unique within the database.
@parameter1 datatype = default_value	- @parameter1: An input parameter for the stored procedure. It starts with @, which is a naming convention for parameters in SQL Server. - datatype: The data type of the parameter (e.g., INT, NVARCHAR, DECIMAL). - = default_value: An optional default value for the parameter. If no value is provided when the procedure is called, this default value will be used.
@parameter2 datatype = default_value	- @parameter2: Another input parameter, following the same structure as @parameter1. - datatype: The data type of the parameter (e.g., INT, NVARCHAR, DECIMAL). - = default_value: An optional default value for the parameter.
AS	- AS: This keyword separates the procedure's declaration (name, parameters) from its body (the SQL logic).
BEGIN	- BEGIN: This keyword marks the start of the procedure's body, where you define the SQL statements to be executed.
SELECT column1, column2 FROM table_name WHERE condition;	- SQL statements: The main logic of the procedure. In this example, a SELECT statement is used to retrieve specific columns from a table based on a condition. - column1, column2: The columns to be selected from the table. - table_name: The name of the table from which data is being selected. - condition: The filtering condition used in the WHERE clause to specify which rows should be returned.
-- Additional SQL logic	- Additional SQL logic: Placeholder for any other SQL operations that the procedure might perform, such as inserts, updates, deletes, or further calculations.
END	- END: This keyword marks the end of the procedure's body, concluding the SQL logic encapsulated within the procedure.

EXAMPLE

```
CREATE PROCEDURE GetEmployeeDetails
@EmployeeID INT
AS
BEGIN
    SELECT FirstName, LastName, Department, Salary
    FROM Employees
    WHERE EmployeeID = @EmployeeID;
END;
```

This procedure, **GetEmployeeDetails**, accepts an employee ID as a parameter and returns the corresponding employee's details.

EXECUTING STORED PROCEDURES

- **Syntax:**

```
EXEC procedure_name @parameter1 = value1, @parameter2 = value2;
```

- **Example:**

```
EXEC GetEmployeeDetails @EmployeeID = 101;
```



This command executes the GetEmployeeDetails procedure with an EmployeeID of 101.

MODIFYING STORED PROCEDURES

■ Syntax:

```
ALTER PROCEDURE procedure_name  
AS  
BEGIN  
    -- New or modified SQL statements  
END;
```

EXAMPLE:

```
ALTER PROCEDURE GetEmployeeDetails  
@EmployeeID INT  
AS  
BEGIN  
    SELECT FirstName, LastName, Department, Salary, HireDate  
    FROM Employees  
    WHERE EmployeeID = @EmployeeID;  
END;
```

This modification adds the **HireDate** column to the GetEmployeeDetails procedure.

DROPPING STORED PROCEDURES

- Syntax:

```
DROP PROCEDURE procedure_name;
```

- Example:

```
DROP PROCEDURE GetEmployeeDetails;
```

This command deletes the GetEmployeeDetails procedure from the database.

TRIGGERS IN SQL

- A trigger is a special type of stored procedure in SQL that automatically executes or "fires" when specific events occur within the database. These events can be actions like INSERT, UPDATE, or DELETE on a table or view.
- Types of Triggers:
 1. **DML Triggers (Data Manipulation Language Triggers)**
 2. **DDL Triggers (Data Definition Language Triggers)**
 3. **Logon Triggers**

TYPES OF TRIGGERS:

1. DML Triggers (Data Manipulation Language Triggers):

- **AFTER Triggers:** Execute after a specified event (like INSERT, UPDATE, or DELETE) has occurred.
- **INSTEAD OF Triggers:** Execute in place of the specified event, overriding the standard action.

2. **DDL Triggers (Data Definition Language Triggers):** Fire in response to DDL statements such as CREATE, ALTER, or DROP that modify the database schema.

3. **Database Trigger (LOGON/ LOGOFF/ STARTUP / SHUTDOWN Triggers):** Fire in response to LOGON/ LOGOFF/ STARTUP / SHUTDOWN events, which are raised when a user session is established.

SYNTAX

```
CREATE TRIGGER [TRIGGER_NAME]  
[TRIGGER_TIME-BEFORE | AFTER] {INSERT | UPDATE | DELETE}  
ON [TABLE_NAME]  
AS  
[TRIGGER_BODY]
```

HOW TO CREATE A TRIGGER? - EXAMPLE

- When a data is inserted into a table display a current date.

```
CREATE TRIGGER TT1  
ON CUSTOMER  
AFTER INSERT  
AS  
PRINT GETDATE()
```

EXAMPLE 2:

- **Question:** Suppose I want to make a table read only.

```
CREATE TRIGGER TrReadOnly ON Customer
INSTEAD OF
    INSERT,
    UPDATE,
    DELETE
AS
BEGIN
    RAISERROR('Customer table is read only.' )
    ROLLBACK TRANSACTION
END
```

Whenever a user executes an INSERT/UPDATE/DELETE statement, the transaction will fail and fire error.
Customer table is read only

PROBLEMS OF TRIGGERS

- Triggers are not maintenance friendly.
 - Bcz, they just “happen” as a side effect of some other operation.
- It is easy to view table relationships , constraints, indexes, stored procedure in database but triggers are difficult to view.
- Triggers run every time when the database fields are updated and it is overhead on system. It makes system run slower.

CASE STUDY

- Suppose a BigBazaar company have a database named BigBazaar_DB, and this database contains 5 tables. One of this table named Sales. In this table daily nearly 500 rows is inserting through website. The table fields are SaleNo, SalesDate, SName, Quantity. Now his requirement is
 1. All data of a month (previous month) should delete on the first day of Every month .
 2. Before deleting table back up should create in one folder with these one month back data.

SOLUTION

1. All the data till 1 month back should delete on the first day of Every month .

```
INSERT INTO Backup  
SELECT SaleNo,SalesDate,SName, Quantity  
FROM Sales  
WHERE  
SalesDate < getdate()
```


SOLUTION

2. Remove all backed up data:

```
DELETE FROM Sales  
WHERE  
SalesDate < getdate()
```

DATABASE SECURITY AND AUTHORIZATION

- **Database Security and Authorization** are essential concepts to protect sensitive data from unauthorized access, breaches, and misuse.
- **1. Database Security:**
 - Database security refers to the range of protective measures that are designed to safeguard a database from threats such as unauthorized access, misuse, or malicious attacks.
 - Securing databases is critical because they often store highly sensitive data such as personal information, financial records, and business-critical information.

DATABASE SECURITY AND AUTHORIZATION

■ Key Aspects of Database Security:

- **Confidentiality:** Ensuring that only authorized users have access to sensitive data.
- **Integrity:** Ensuring that the data remains accurate and consistent over its lifecycle.
- **Availability:** Ensuring that authorized users can access the database when required.

DATABASE SECURITY AND AUTHORIZATION

■ Common Database Threats:

- **SQL Injection Attacks:** Malicious SQL queries are injected into an application's input fields to manipulate or compromise the database.
- **Privilege Abuse:** When users misuse their access rights to view or manipulate data beyond their assigned permissions.
- **Data Breaches:** Unauthorized access leading to the exposure or theft of sensitive information.
- **Denial of Service (DoS):** Attacks aimed at making the database unavailable by overwhelming the system with excessive requests.

DATABASE SECURITY AND AUTHORIZATION

■ Techniques to Enhance Database Security:

1. Encryption:

1. Sensitive data can be encrypted both in transit (as it travels over networks) and at rest (when stored).
2. Database-level encryption ensures that even if data is accessed, it remains unreadable without the decryption key.

2. User Authentication:

1. Authentication ensures that only authorized individuals can access the database.
2. Techniques include password-based authentication, biometric methods, two-factor authentication (2FA), and single sign-on (SSO).

DATABASE SECURITY AND AUTHORIZATION

3. Access Controls (Authorization):

- Define which users or systems have access to specific data within the database.
- Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) are commonly used models.

4.Regular Audits (Auditing and Logging):

- Database audit logs track who accessed the database, what actions they performed, and when.
- Regular audits help identify suspicious activities, anomalies, or access patterns.

5.Firewalls and Intrusion Detection Systems:

- Database firewalls prevent SQL injection attacks by filtering out harmful queries.
- Intrusion Detection Systems (IDS) monitor network traffic for suspicious activities.

DATABASE SECURITY AND AUTHORIZATION

6. Backup and Recovery:

6. Maintaining secure backups is essential for recovering from data loss incidents or attacks.
7. Backup systems should be secure, encrypted, and tested regularly to ensure availability when needed.

DATABASE AUTHORIZATION

- Authorization is a subset of database security and refers to the process of granting or restricting access to resources based on a user's identity and their assigned permissions.
- It determines *who* can access *what* data and *how* they can interact with that data (e.g., read, write, delete).
- **Types of Authorization Mechanisms:**
 - Discretionary Access Control (DAC)
 - Mandatory Access Control (MAC)
 - Role-Based Access Control (RBAC)
 - Attribute-Based Access Control (ABAC)

TYPES OF AUTHORIZATION MECHANISMS

- **Discretionary Access Control (DAC):** Access is determined by the database owner, who has the discretion to grant permissions to other users. For example, the owner of a table can grant read or write access to specific users.
- **Mandatory Access Control (MAC):** Access is controlled at a system-wide level and is based on classifications (such as Top Secret, Confidential). Users have access based on their clearance levels, and information is accessed according to strict policies.

TYPES OF AUTHORIZATION MECHANISMS

- **Role-Based Access Control (RBAC):** Access rights are assigned to roles, and users are assigned roles based on their job functions. This simplifies management since permissions are managed at the role level rather than for individual users.
- **Attribute-Based Access Control (ABAC):** Access is determined by evaluating attributes associated with users, resources, and the environment. Policies dictate access based on user characteristics (e.g., job title, department) or environmental conditions (e.g., time of day).

KEY CONCEPTS IN AUTHORIZATION:

- **Privileges:** Privileges define what actions users can perform on specific objects (e.g., SELECT, INSERT, DELETE, UPDATE). Privileges can be assigned at various levels such as the database, table, or column level.
- **Grant and Revoke:**
 - Grant: This command gives a user or role certain privileges to access or modify data in the database.
 - Revoke: This command removes previously granted privileges.

KEY CONCEPTS IN AUTHORIZATION:

- **Roles and Role Hierarchies:** Roles simplify the management of permissions. Rather than assigning permissions to individual users, roles (such as “Admin,” “User,” or “Manager”) are assigned to users. Role hierarchies can be created where higher-level roles inherit permissions from lower-level roles.

WAYS TO DATABASE SECURITY AND AUTHORIZATION

- **Implement Least Privilege:** Grant users the minimum level of access required to perform their tasks, reducing the risk of accidental or intentional misuse.
- **Use Strong Password Policies:** Enforce strong password policies (e.g., minimum length, complexity requirements, expiration periods).
- **Apply Patching and Updates:** Regularly apply security patches and updates to the database and related systems to protect against vulnerabilities.
- **Segregate Sensitive Data:** Store sensitive data separately from non-sensitive data, applying stronger security controls.
- **Use Multifactor Authentication (MFA):** MFA adds an extra layer of protection by requiring more than one form of verification to access the database.
- **Monitor and Respond to Incidents:** Continuously monitor access and activities, and have a clear incident response plan in place for dealing with security breaches.